

Experiment 4: Web API CRUD Operations

Aim

To perform Create, Read, Update, and Delete (CRUD) operations on Employee data using ASP.NET Core Web API and test the API using Swagger.

Step 1: Open the Existing Project

1. Open **Visual Studio 2022**.
 2. Click **Open a project or solution**.
 3. Select your **SwaggerDemo** project (created in the previous experiment).
 4. Wait until the project loads completely.
-

Step 2: Build the Project

1. Click **Build** → **Build Solution**.
 - Or press **Ctrl + Shift + B**.
2. Ensure the Output window displays:

Build succeeded.

Step 3: Open EmployeeController

1. In **Solution Explorer**, expand the **Controllers** folder.
 2. Double-click **EmployeeController.cs**.
-

Step 4: Verify Existing CRUD Methods

Ensure your EmployeeController contains the following methods:

- `GetStandard()` (GET)
- `Post()` (POST)
- `Put()` (PUT)
- `Delete()` (DELETE)

If they are already present, proceed to the next step.

Step 5: Modify the PUT Method

Replace your existing Put() method with the updated version that performs validation.

Validation Rules

1. If id <= 0, return:

```
return BadRequest("Invalid employee id");
```

2. Search the employee list:

```
var existingEmployee = employees.FirstOrDefault(e => e.Id == id);
```

3. If no employee is found, return:

```
return BadRequest("Invalid employee id");
```

4. If the employee exists:

- Update Name
- Update Salary
- Update Permanent
- Update Department
- Update Skills
- Update DateOfBirth

5. Return the updated employee using:

```
return Ok(existingEmployee);
```

Step 6: Modify the DELETE Method

Replace your existing Delete() method with the updated version.

Validation Rules

1. If id <= 0, return:

```
return BadRequest("Invalid employee id");
```

2. Search for the employee:

```
var employee = employees.FirstOrDefault(e => e.Id == id);
```

3. If no employee is found, return:

```
return BadRequest("Invalid employee id");
```

4. If the employee exists:

- Remove it from the list.

- Return:

```
return Ok("Employee Deleted Successfully");
```

Step 7: Save the Project

1. Click **File** → **Save All**.
 - Or press **Ctrl + Shift + S**.
-

Step 8: Build the Project Again

1. Press **Ctrl + Shift + B**.
 2. Verify the build succeeds without errors.
-

Step 9: Run the Project

1. Press **Ctrl + F5**.
2. The browser opens automatically.
3. Swagger UI appears at:

<https://localhost:xxxx/swagger>

Step 10: Test the GET Method

1. Expand **Employee**.
2. Click **GET**.
3. Click **Try it out**.
4. Click **Execute**.

Expected Result

- Employee details are displayed.
- Status Code:

200 OK

Step 11: Test the POST Method

1. Expand **POST**.

2. Click **Try it out**.
 3. Enter Employee details in JSON format.
 4. Click **Execute**.
-

Step 12: Test the PUT Method

Test Case 1 – Invalid ID

1. Expand **PUT**.
2. Click **Try it out**.
3. Enter:

id = -1

4. Enter any valid Employee JSON.
5. Click **Execute**.

Expected Result

400 Bad Request

Invalid employee id

Test Case 2 – Employee Not Found

1. Enter:

id = 100

2. Click **Execute**.

Expected Result

400 Bad Request

Invalid employee id

Test Case 3 – Valid Employee

1. Enter:

id = 1

2. Enter the updated Employee JSON.
3. Click **Execute**.

Step 13: Test the DELETE Method

Test Case 1 – Invalid ID

1. Expand **DELETE**.
2. Click **Try it out**.
3. Enter:

id = -1

4. Click **Execute**.

Expected Result

400 Bad Request

Invalid employee id

Test Case 2 – Employee Not Found

1. Enter:

id = 100

2. Click **Execute**.

Expected Result

400 Bad Request

Invalid employee id

Test Case 3 – Valid Employee

1. Enter:

id = 1

2. Click **Execute**.

Expected Result

Employee Deleted Successfully

Status Code:

200 OK

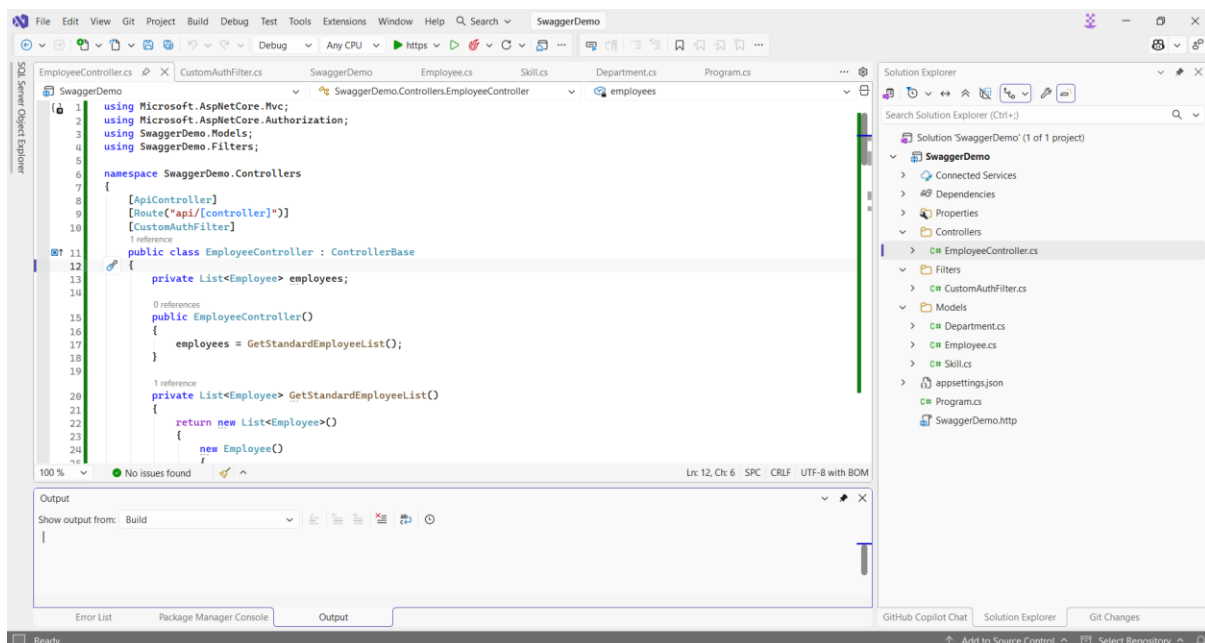
Step 14: Test Using Postman (Optional)

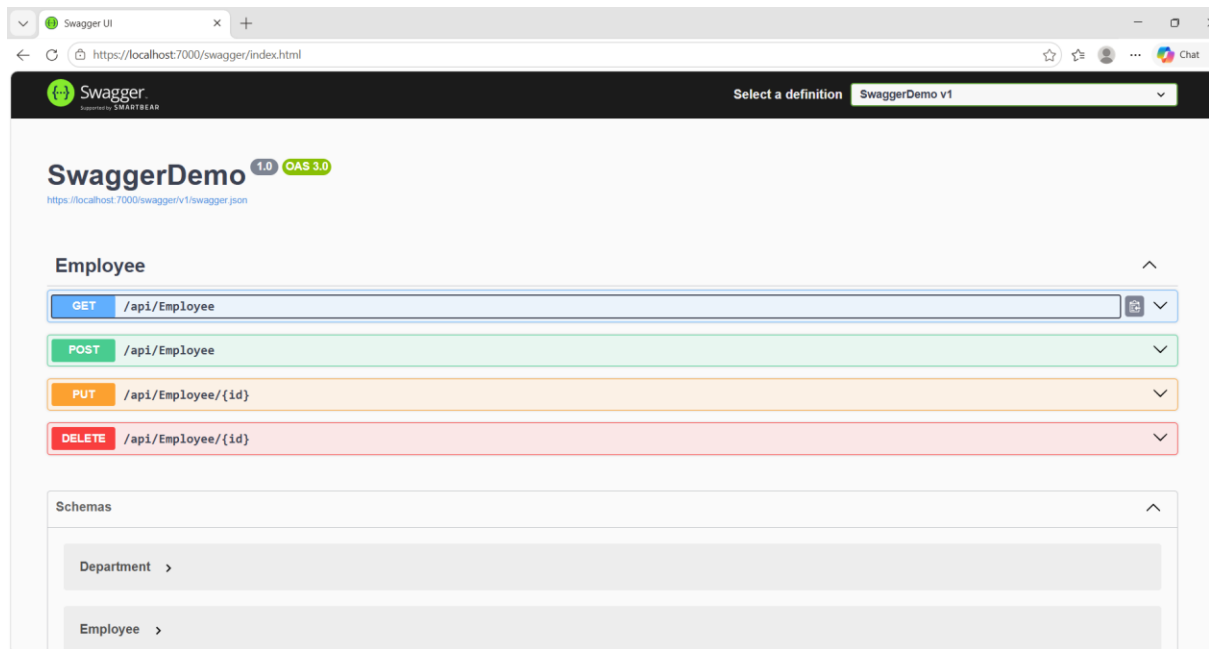
1. Open **Postman**.
2. Create a new request.
3. Select:
 - **GET, POST, PUT,**
 - or **DELETE.**
4. Enter the URL:
`https://localhost:xxxx/api/Employee`
5. For POST and PUT:
 - Select **Body** → **raw** → **JSON**.
 - Enter Employee JSON.
6. Click **Send**.
7. Verify the response and status code.

Step 15: Result

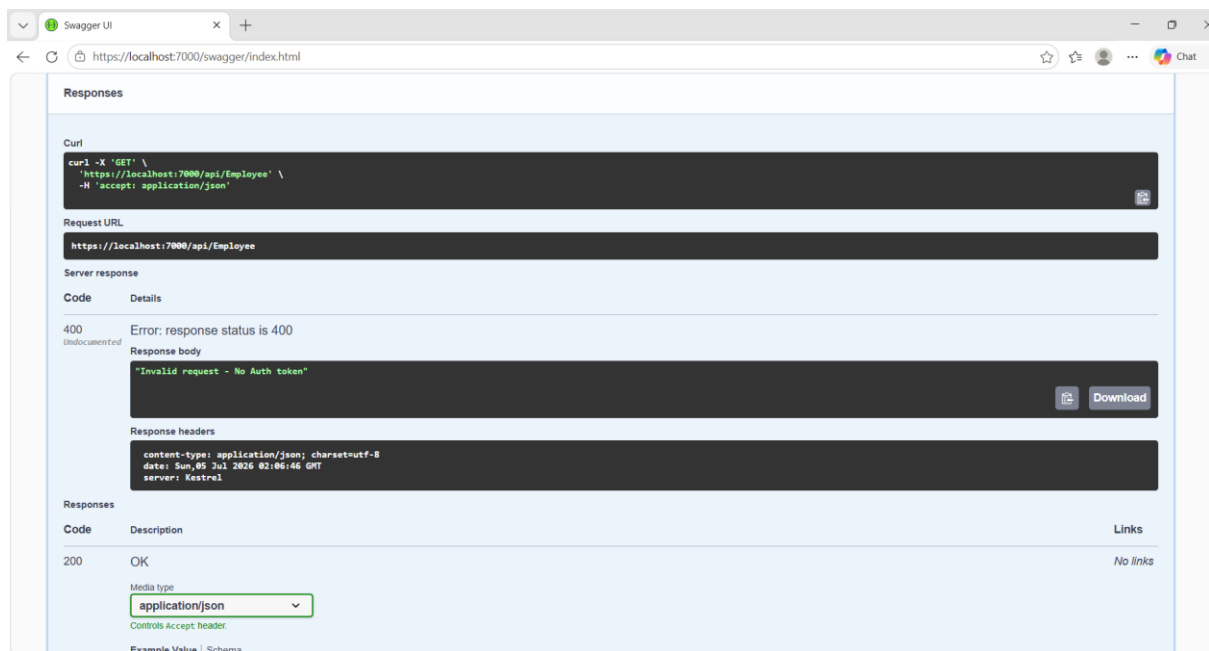
Successfully implemented CRUD operations in an ASP.NET Core Web API. The API validates employee IDs, updates employee details using the PUT method, deletes employee records using the DELETE method, and was successfully tested using Swagger and Postman.

Outputs:





GET METHOD:



POST METHOD:

Swagger UI

https://localhost:7000/swagger/index.html

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7000/api/Employee' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 1,
    "name": "John",
    "salary": 50000,
    "permanent": true,
    "department": {
      "id": 101,
      "name": "IT"
    },
    "skills": [
      {
        "id": 1,
        "name": "C#"
      },
      {
        "id": 2,
        "name": "SQL"
      }
    ],
    "dateOfBirth": "1998-05-10T00:00:00"
  }'
```

Request URL

https://localhost:7000/api/Employee

Server response

Code	Details
400	Error: response status is 400

Response body

```
Invalid request - No Auth token
```

Download

PUT METHOD:

Swagger UI

https://localhost:7000/swagger/index.html

Curl

```
curl -X 'PUT' \
  'https://localhost:7000/api/Employee/1' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 1,
    "name": "John Updated",
    "salary": 70000,
    "permanent": true,
    "department": {
      "id": 101,
      "name": "IT"
    },
    "skills": [
      {
        "id": 1,
        "name": "ASP.NET Core"
      },
      {
        "id": 2,
        "name": "SQL Server"
      }
    ],
    "dateOfBirth": "1998-05-10T00:00:00"
  }'
```

Request URL

https://localhost:7000/api/Employee/1

Server response

Code	Details
400	Error: response status is 400

Response body

```
Invalid request - No Auth token
```

Response headers

```
content-type: text/plain; charset=utf-8
date: Sun, 05 Jul 2026 02:18:22 GMT
server: Kestrel
```

Download

DELETE METHOD:

Swagger UI

+

https://localhost:7000/swagger/index.html

Curl

curl -X 'DELETE' \

'https://localhost:7000/api/Employee/1' \

-H 'accept: */*'

Request URL

https://localhost:7000/api/Employee/1

Server response

Code	Details
400	Error: response status is 400

Undocumented

Response body

Invalid request - No Auth token

Response headers

content-type: text/plain; charset=utf-8

date: Sun, 05 Jul 2026 02:11:06 GMT

server: Kestrel

Responses

Code	Description	Links
200	OK	No links

Schemas